

ORIGINAL ARTICLE OPEN ACCESS

CryptoCloud: A Usable Framework for Storing, Searching and Sharing Encrypted Data on Cloud Storage

Md Mazharul Islam  | Md. Shahadul Alam Patwary | Salekul Islam  | Rajesh Palit 

¹ Department of Electrical and Computer Engineering, North South University, Dhaka, Bangladesh |

Correspondence: Salekul Islam. salekul.islam@northsouth.edu

Received: Date Month Year | **Revised:** Date Month Year | **Accepted:** Date Month Year

Keywords: Cloud storage, data confidentiality, encrypted search, key management, privacy preservation, searchable encryption, secure data sharing.

Abstract

Security has become a significant concern with the increased popularity of cloud storage services. It comes with the vulnerability of being accessed by third parties. It has given rise to security concerns in data confidentiality and unauthorized access, even after the deletion of data from cloud storage. This becomes a severe issue when the replicated copies remain hidden in the server even after the owner has deleted the original file. Thus, data needs to be encrypted before uploading to the public cloud. However, it creates a hindrance when the encrypted data needs to be shared with third parties without any already established trusted relationship. Besides, searching, a crucial functionality in cloud storage, does not work on any encrypted data. Searchable encryption (SE) allows a cloud server to conduct a search over encrypted data on behalf of the data users without learning the stored data. While many SE schemes show provable security, they expose query information and do not provide effective data utilization. Also, sharing encrypted data with other authorized users must ensure providing a document-specific secret key, creating massive key management overhead. To address the above issues, we design CryptoCloud, a secure and searchable encryption framework built on an encrypted inverted index, using symmetric cryptography for search and public-key cryptography solely for secure key distribution. We implement the system using widely used cryptographic techniques, ensuring search on encrypted data. The primary focus is to ensure user privacy and security through a less computationally intensive, user-friendly system with a trusted third-party entity. The system authenticates users through this entity and issues encrypted search keywords, ensuring identical queries remain unlinkable to the cloud. Its encrypted inverted index supports efficient single and multi-keyword search, while avoiding per-document key distribution to minimize the key-management overhead. All the cryptographic operations are performed at the trusted third-party and client sides, so plaintext and keys are never exposed to the storage provider. We also develop a web application as an overlay system of CryptoCloud on a cloud storage domain. Our performance study demonstrates the low response time of our system while maintaining scalability.

1 | Introduction

Cloud computing has reshaped how data is stored and processed by offering elastic, on-demand access to shared, configurable resources over the Internet [1]. Built on advances in virtualization, autonomic and grid computing, and service-oriented architectures, today's cloud platforms support a wide range of consumer and enterprise services across government and industry. The model reduces capital expenditure, simplifies operations,

and enables collaboration from any location. Service layers—Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS) provide modular service delivery, while Storage as a Service exposes affordable object storage to end users and applications. A widely cited definition from NIST describes cloud computing as a pay-per-use model that offers convenient, on-demand network access to shared resources which can be rapidly provisioned and released with minimal management effort [2].

Notwithstanding these benefits, there are serious issues with confidentiality, integrity, lifecycle control, and auditability when data is outsourced to outside cloud service providers [3]. Replicated copies may persist after deletion, and users no longer have direct control over their data or encryption keys. Furthermore, trustworthy but inquisitive suppliers or external adversaries can try to deduce private data from usage patterns or archived content. In reality, there are still significant barriers to broader adoption for sensitive workloads, including insider risks, multi-tenant leakage, compliance and governance constraints, and performance management [4]. Storage needs confidentiality, integrity, access control, and guaranteed erasure, whereas compute needs privacy-preserving and verifiable processing when the cloud performs queries or analytics on users' behalf.

Since it prevents providers from learning plaintext data even if their infrastructure is compromised, client-side encryption is a logical first line of defense [5]. While public-key cryptography enables secure distribution and fine-grained control, symmetric-key cryptography offers efficiency with a shared secret. However, there is a usability gap when data is encrypted before upload, since the server-side cannot perform direct keyword searches or selective retrieval after encryption. Security is compromised when decryption keys are shared with the provider, and it is not cost-effective to download and locally decode the entire dataset [6]. The tension between confidentiality and usability requires mechanisms that protect data privacy while enabling efficient server-side search and retrieval.

This problem is addressed by searchable encryption (SE), which enables keyword queries over ciphertexts without disclosing the query terms or the document content to the cloud provider [7]. Before outsourcing documents to the cloud, the data owner in a standard SE system encrypts them along with a searchable index. Only the pertinent ciphertexts are returned by the cloud once authorised users create encrypted query tokens, also known as trapdoors, which are compared to the encrypted index. This research topic includes both symmetric searchable encryption and public-key encryption, each with trade-offs among efficiency, expressiveness, and trust assumptions. Indexing techniques often resemble plaintext retrieval: forward indexes map documents to keywords, whereas inverted indexes map keywords to documents [8]. Because of their efficient sublinear search capabilities, encrypted inverted indexes are commonly used in large-scale systems. Large-scale systems often use encrypted inverted indexes because they offer effective sublinear search capabilities.

Over the last twenty years, searchable encryption has changed dramatically, moving from static datasets to methods that enable regular updates [9] and from single-keyword search to multi-keyword, ranked, and dynamic query models. Minimising information leakage, verifying results, and facilitating multi-user access control have all been investigated in more recent work. Several obstacles still stand in the way of a realistic implementation of public cloud storage, notwithstanding these advancements. Many approaches hinder scalability and usefulness in real-world scenarios by requiring additional trusted infrastructure, imposing significant preprocessing or query cost, or depending on intricate cryptographic primitives. It is crucial to consider query unlinkability, as deterministic query tokens can reveal search patterns [10]. Effective multi-user sharing and revocation remain challenging without necessitating data re-encryption or incurring high key-management costs.

Even when data is protected at rest, practical cloud deployments impose system-level requirements that go beyond ciphertext confidentiality: search must be sublinear over public storage, revocation must take effect immediately, and sharing with new users must not force data re-encryption or per-document key exchange. Recent leakage-hardened searchable encryption sharpens security along individual axes—forward/backward privacy in dynamic SSE [11], size and access-pattern hiding via padding, volume hiding, or ORAM [12] but each such guarantee is purchased with persistent client-side state, auxiliary trusted infrastructure, or super-linear overhead that erodes exactly these deployability properties. Conversely, systems that remain easy to deploy over unmodified cloud storage typically expose deterministic tokens and access patterns. Concretely, we target a framework that (i) runs as an overlay on an unmodified public cloud (Google Drive) with no client-side index state, (ii) supports non-interactive enrollment and immediate revocation for multiple users, and (iii) keeps all plaintext and keys off the provider—while accepting a clearly stated and bounded leakage profile as the price of this deployability.

This study presents a workable system for sharing, storing, and searching encrypted data in untrusted cloud storage. Before being outsourced to the cloud, the suggested method encrypts the documents and the inverted index from beginning to end. Enforcing authorisation, authenticating users, and providing encrypted search terms that render identical queries unlinkable are all handled by a lightweight trusted entity. The cloud provider does not know the content of documents or the search terms being used; it only works with ciphertexts. Only authorised users are granted access to the decryption keys using their own public keys; the supplier never keeps them. It allows for instant revocation without requiring re-encryption of stored data. The main contributions of this work are summarized as follows:

- **Secure multi-user searchable encryption framework:** The proposed framework enables fine-grained keyword search for multiple authorized users while preserving data confidentiality. Search keywords are issued through an authorization mechanism that ensures only legitimate users can perform encrypted queries, maintaining strict access control.
- **Dynamic and privacy-preserving encrypted inverted index:** A secure and dynamic encrypted inverted index is designed to support efficient single- and multi-keyword searches while preventing the leakage of keywords and document contents. The index supports flexible updates and scalable encrypted storage management.
- **Lightweight non-interactive search authorization:** After authorization, encrypted search permissions are generated in a non-interactive manner, supporting dynamic user enrollment without affecting existing users and incurring minimal computation and communication overhead.
- **Deployable overlay architecture:** Rather than a new cryptographic primitive, the contribution is a system design that composes well-understood primitives (AES, RSA) with a TTP-mediated authorization layer to deliver non-interactive multi-user search, instant revocation, and zero provider-side trust as a practical overlay on unmodified cloud storage.

The remainder of this paper is organized as follows. Section 2 presents a review of existing research on secure cloud storage and searchable encryption techniques. Section 3 defines the system and threat models, followed by the security requirements in Section 4. Section 5 describes the proposed CryptoCloud framework and its operational design. Section 6 details the system implementation, and Section 7 evaluates its performance. Section 8 provides a security analysis and formal verification results. Finally, Section 9 concludes the paper and outlines future research directions.

2 | Literature Review

Searchable encryption is now a fundamental component of secure cloud storage systems, enabling keyword-based retrieval of encrypted data stored on untrusted servers. Public-Key Encryption with Keyword Search (PEKS) and Searchable Symmetric Encryption (SSE) are the two main cryptographic paradigms that have shaped this field's development over the last 20 years. To increase efficiency and scalability, indexing strategies in particular, encrypted inverted indexes have been developed concurrently. The representative works in these areas are reviewed in this section, which also places our framework in this context.

2.1 | Secure Cloud Storage and Early Searchable Encryption

Early studies on safe cloud storage focused more on data sharing, secrecy, and guaranteed destruction than on effective retrieval. To protect data from unreliable providers, Patwary and Palit [13] focused on client-side encryption in their prototype system for safe file sharing and storage in cloud storage infrastructures. Without the need for outside, trusted key managers, their strategy reduced risks such as unauthorized access and the permanence of copied data. However, it was not practicable for large-scale datasets where selective keyword-based retrieval is crucial because, like many early secure storage systems, it required full file download and decryption for access.

The first practical attempt to enable search over encrypted data was introduced by Song *et al.* [14], who demonstrated that keyword search could be performed directly on ciphertext without revealing plaintext to the server. Their scheme embeds encrypted keyword information into ciphertext blocks and allows the server to test for keyword presence using a user-generated trapdoor. While this work established the feasibility of searchable encryption and provided strong confidentiality guarantees, it relied on linear scanning of all encrypted documents and did not support indexed search, resulting in poor scalability for cloud environments.

To address efficiency and formal security modeling, Curtmola *et al.* [15] introduced Searchable Symmetric Encryption (SSE) with explicit leakage definitions and efficient constructions based on encrypted inverted indexes. Their work clearly distinguished between encryption security and search security by formalizing access-pattern and search-pattern leakage, demonstrating that sublinear and practical search over encrypted data is achievable. This contribution laid the foundation for modern SSE systems but remained focused on private-key settings and did not consider usability, authorization workflows, or real-world cloud integration.

In parallel, Boneh *et al.* [16] proposed Public-Key Encryption with Keyword Search (PEKS) to support searchable encryption in open and multi-sender environments. PEKS enables keyword encryption using public keys and search using private-key-generated trapdoors, making it suitable for applications such as encrypted email routing. However, PEKS schemes rely on expensive public-key operations, leak access patterns by default, and are vulnerable to keyword-guessing attacks. Extensions incorporating fine-grained access control using attribute-based encryption, such as the scheme proposed by Chaudhari and Das [17], improve expressiveness but introduce heavy computational overhead and complex trust assumptions.

Islam and Palit [18] built on previous work by introducing a keyword-based search-and-share system for encrypted cloud data that uses an encrypted inverted index and symmetric encryption. That work demonstrated that safe keyword search and restricted sharing were possible in a limited setting. It enabled both single and multi-keyword searches and reduced unnecessary downloads, but it failed to address formal threat modelling, system architecture, and scalability. The current work builds on that initial concept by providing a formal security architecture, an improved authorisation and revocation mechanism, and a thorough end-to-end implementation suitable for actual cloud storage settings.

2.2 | Searchable Symmetric Encryption (SSE)

The goals of recent research on searchable symmetric encryption (SSE) have been to facilitate dynamic updates, improve efficiency while maintaining usability in cloud environments, and strengthen security against practical inference attacks. One important area of research examines how adversaries can exploit leaks that occur during encrypted searches and proposes defences against such threats. Li *et al.* [19] examined response identity attacks, in which adversaries can deduce sensitive information about underlying keywords by repeatedly seeing identical query responses. They provide a feasible SSE design that integrates controlled randomisation into search results to lessen the adversary's capacity to correlate related queries while maintaining efficient search performance. Although the system is successful in limiting answer identity leaks, access patterns are still revealed, and long-term query behaviour analysis remains incomplete.

A substantial amount of research also examines query correlation and its impact on SSE security, especially for conjunctive queries. Liu *et al.* [20] showed that correlations between encrypted query tokens can allow for extremely successful query recovery attacks, even when conventional leakages like access and search patterns are reduced. Their work presents a passive approach that produces high keyword recovery rates in practical scenarios and formalises generalised query correlation patterns for conjunctive SSE. Stronger unlinkability methods or query obfuscation techniques are required, as this conclusion emphasises that leakage from user query behaviour itself is a fundamental challenge for SSE.

Several works suggest strong dynamic SSE methods with formal forward and backward security assurances to facilitate dynamic data outsourcing. RO(SE)² is a search-efficient and resilient SSE technique that guarantees both forward and backward security while allowing for update failures and inconsistencies, as presented by Yang *et al.* [11]. Resilience is a crucial component of

their design as updates can be halted or reorganised in real-world cloud environments. Parallel to this, Bian *et al.* [21] developed dynamic SSE to support conjunctive queries, proposing techniques that make expressive search possible while preserving update security. By implementing a lightweight update search permission control, SEAC [22] further improved this strategy by enabling fine-grained authorisation changes without requiring significant cryptographic overhead. Although these techniques significantly improve practicality, they still reveal some access pattern information and require maintaining additional client-side data.

Research in SSE is still ongoing in the areas of expressive search and leakage reduction. By adding padding and structural modifications to conceal result sizes, SMSSE [12] explicitly addressed size-pattern leakage, which exposes information through the quantity of returned results. A leakage-reduced SSE approach for multi-keyword queries was suggested by Deng *et al.* [23]. This scheme uses carefully crafted index and token structures to minimise information exposure during conjunctive search. A privacy-preserving conjunctive SSE technique, designed for Cloud-IoT healthcare systems, was developed by Ma *et al.* [24] for application-oriented contexts, striking a balance between efficiency and secrecy in a field with stringent privacy regulations. These strategies reduce specific leakage pathways, but they often entail additional storage or processing costs.

Lastly, several studies investigate orthogonal improvements to SSE, such as decentralisation, verifiability, and efficiency optimisation. By optimising index representations and cryptographic operations, Chakraborty *et al.* [25] concentrated on making SSE schemes quicker and smaller while also obtaining significant savings in computation and storage expenses. Although it came at the expense of controlled false positives, BloomSec [26] used Bloom-filter-based indexes to increase scalability and privacy in cloud environments. Verifiable SSE was introduced by Ji *et al.* [27] utilising additive homomorphic encryption to enable users to confirm that search results from an untrusted server are accurate. SEARCHAIN [28] combined SSE with blockchain-based rewarded practical work to offer decentralisation and auditability, but at the expense of higher deployment overhead and system complexity. Together, these studies demonstrate significant advances in SSE research while highlighting ongoing challenges in balancing expressiveness, efficiency, and leakage resilience.

2.3 | Public-Key Encryption with Keyword Search (PEKS)

Public-Key Encryption with Keyword Search (PEKS) has evolved into a rich research area aimed at enabling keyword-based search over encrypted data in open, multi-user environments where shared secret keys are impractical. Unlike symmetric searchable encryption, PEKS allows data senders to encrypt keywords using a receiver's public key, enabling any authorized receiver to search encrypted data via trapdoors generated from their private key. While this paradigm naturally supports decentralized data sharing, extensive research has shown that PEKS schemes are inherently vulnerable to keyword guessing attacks (KGA), particularly when the keyword space is small. As a result, much of the PEKS literature focuses on mitigating both outside and insider KGAs through authenticated encryption, designated testers, proxy assistance, and verifiability.

One major research direction strengthens PEKS through authentication and tester restriction. Public-key authenticated encryption with keyword search (PAEKS) requires the sender's secret key during keyword encryption, ensuring that even an insider cloud server cannot mount dictionary attacks. However, authenticated PEKS schemes significantly increase computation and trapdoor size in multi-sender settings. To address this, proxy-assisted approaches were introduced, where a trusted proxy converts heterogeneous ciphertexts into a uniform format to enable constant-size trapdoors. More recently, proxy-free authenticated PEKS schemes eliminate reliance on trusted intermediaries by allowing the cloud server to securely update ciphertexts itself, achieving efficient search while resisting insider KGAs [29]. Lightweight authenticated PEKS constructions further reduce overhead for multi-user cloud storage, though they still rely on expensive public-key primitives [30].

Another active line of work focuses on dynamic, verifiable, and fine-grained PEKS. Dynamic PEKS schemes support document insertion and deletion without rebuilding encrypted indexes, improving applicability to real cloud systems [31]. Verifiable PEKS allows users to confirm the correctness of search results returned by an untrusted server, often using homomorphic tags or pairing-based proofs [27, 32]. However, recent crypt-analytic studies reveal subtle weaknesses in designated-tester PEKS designs: security analyses demonstrate that pairing-based constructions relying on symmetric pairings may invalidate decisional Diffie–Hellman assumptions, leading to keyword leakage through trapdoor linkability or tester key exposure [33]. These results highlight the fragility of PEKS security when strong trust assumptions on servers or testers are relaxed.

Lattice-based PEKS methods have been developed to provide post-quantum security to address long-term security issues. These designs support advanced features such as multi-user authentication, ciphertext updating, and forward security by substituting the Learning With Errors (LWE) assumption for bilinear pairings [34]. While lattice-based PEKS significantly strengthens cryptographic guarantees, it incurs substantial ciphertext expansion, complex key management, and high computational cost. Even efficiency-oriented PEKS schemes based on arithmetic span programs or optimized public-key operations remain several orders of magnitude slower than symmetric approaches under high query volumes [25]. As a result, PEKS algorithms are unable to adequately meet the latency and scalability requirements of real-world cloud storage systems that often offer multi-keyword searches. These efficiency limitations highlight the fundamental trade-off between robust cryptographic expressiveness and realistic deployment performance. Because of this, symmetric searchable encryption remains preferred in many real-world cloud storage applications that require low latency and high query throughput.

2.4 | Encrypted Inverted Index Techniques

In searchable encryption systems, encrypted inverted indexes are the fundamental data structure that enables effective keyword-based retrieval. Inverted indexes map keywords to document IDs, making them especially appropriate for large datasets with high query volumes, whereas forward indexes link documents to keyword sets. The idea of secure indexes utilising Bloom filters

to provide encrypted keyword testing with lower storage overhead was first presented in early work by Goh [35]. Although this method outperformed ciphertext scanning in search performance, it had limitations regarding dynamic updates and false positives. Because of its predictable lookup efficiency and compliance with real-world cloud storage infrastructures, encrypted inverted lists became the predominant indexing approach in later SSE designs.

Inverted index designs were developed to accommodate Boolean and multi-keyword searches as encrypted search capabilities got more powerful. To save search space while maintaining privacy, recent systems use layered or hybrid indexing structures. By integrating classification trees, Bloom filters, and encrypted inverted lists, Fugkeaw *et al.* [36, 37] proposed hybrid inverted index frameworks that efficiently enable Boolean searches across encrypted cloud logs and collaborative IIoT data. By eliminating unnecessary ciphertexts early in the search procedure, these methods lower query complexity. However, they add additional information structures that might increase leakage surfaces and rely on auxiliary trust mechanisms like smart contracts or blockchain-based authorisation.

Another area of research adds decentralisation and verifiability to encrypted inverted indexes. Blockchain-assisted searchable encryption systems allow for tamper detection and result verification by embedding encrypted inverted indexes within distributed ledgers or off-chain storage that is anchored by on-chain commitments [38]. These methods improve auditability and transparency, but due to consensus processes and on-chain activities, they come with significant communication costs and delays. Furthermore, it is still tricky to dynamically maintain encrypted inverted indexes in decentralised environments, especially when allowing for high query concurrency, frequent updates, or revocation.

In addition to functionality, new research focuses on measuring and reducing the information leakage that inverted index-based searchable encryption inherently causes. Ahn *et al.* [39] showed that response-size leakage from inverted index queries can allow keyword inference attacks even when access and search patterns are concealed using ORAM (Oblivious RAM). They demonstrated that leakage is a continuum rather than a binary trait by using a quantitative tool to assess the efficacy of volume hiding. Many current systems rely on complex auxiliary infrastructure, heavyweight cryptographic techniques, or additional trust assumptions, even though encrypted inverted indexes remain the most feasible basis for effective searchable encryption in cloud contexts. The need for lightweight inverted index designs that maintain efficiency while carefully controlling leakage and operational complexity is driven by these difficulties.

2.5 | Discussion and Positioning

The aforementioned review demonstrates notable developments in searchable encryption spanning symmetric, public-key, and index-based approaches. Although Searchable Symmetric Encryption (SSE) delivers exceptional efficiency and scalability by using encrypted inverted indexes and dynamic updates, it exposes access patterns, response sizes, and query correlations, leaving the system vulnerable to inference attacks. Techniques like padding, volume hiding, or ORAM-based protection reduce

leaks but incur a computational and storage cost, limiting their scalability under high concurrency.

In open environments, Public-Key Encryption with Keyword Search (PEKS) provides stronger cryptographic guarantees and flexible access control. Nevertheless, lattice-based and pairing-based systems entail high computational costs, large ciphertext and trapdoor sizes, and challenging key management. Even optimized or proxy-free variants remain substantially less efficient than symmetric approaches, making them less suitable for frequent encrypted searches in user-centric cloud storage systems. The most practical basis for searchable encryption remains encrypted inverted indexes, thanks to their sublinear search efficiency and compatibility with public cloud storage. However, structural leakage remains a problem, as existing solutions often add layers of verifiability, blockchain integration, or new trust assumptions, thereby increasing system complexity.

In this context, our proposed framework, CryptoCloud, is positioned as a deployable, lightweight alternative. To achieve effective search while preserving controlled leakage under a realistic threat model, it uses an SSE-based encrypted inverted index design. Unlike PEKS-heavy systems, public-key cryptography is used only for secure key distribution, reducing search-time overhead. The framework enables smooth interaction with existing cloud storage providers by avoiding the use of trusted execution environments and decentralised infrastructure, unlike blockchain-assisted techniques. Table 1 shows how CryptoCloud provides a practical compromise for real encrypted cloud storage by balancing usability, security, and efficiency.

We emphasize that CryptoCloud's contribution is architectural rather than primitive-level. Schemes such as VKSE-MO [40], which introduces multisignature-based result verification for multi-owner search, advance the underlying cryptography; our goal is orthogonal and complementary. We deliberately restrict ourselves to standardized, hardware-accelerated primitives (AES, RSA) so that the entire framework is auditable, dependency-free, and directly deployable over public cloud storage without trusted execution environments, blockchains, or bespoke assumptions. Because the index remains purely symmetric and all public-key cost is confined to occasional key distribution, per-query search cost stays independent of the number of owners or verifiers. The novelty lies in how these primitives are composed non-interactive TTP-issued search tokens, per-file key wrapping, and immediate revocation—to close the deployability gap.

3 | Threat Model

For the threat model of our proposed framework, we make the following assumptions: the Cloud Service Provider (CSP), the Trusted Third Party (TTP), the Data Owner (DO), the Data User (DU), and an Adversary (AD). Before being stored in the CSP, the DO encrypts the data and its associated indexes. With permission, the DU requests information from the CSP and retrieves encrypted documents. The TTP is responsible for storing the keys and managing authorization. The CSP provides storage and computational resources but is not trusted. It is modeled as an honest-but-curious adversary that follows prescribed protocols but may attempt to infer sensitive information from encrypted files, indexes, or queries. At no point does the CSP have access

TABLE 1 | Comparison of representative searchable encryption schemes most relevant to this work

Scheme	SE Type	Index	Query	Dynamics	Third Party	Key Limitation
Curtmola <i>et al.</i> [15]	SSE	Inv-Enc	Single	Static	None	No updates, leakage
Yang <i>et al.</i> [11]	DSSE	Inv-Enc	Single	F/B	None	Client state, overhead
Bian <i>et al.</i> [21]	DSSE	Inv-Enc	Conjunctive	F/B	None	Query correlation leakage
SMSSE [12]	SSE	Inv-Enc	Multi	Static	None	Padding overhead
BloomSec [26]	SSE	Bloom + Inv	Single	Static	None	False positives
SEARCHAIN [28]	SSE + BC	Inv-Enc	Multi	Static	Blockchain	High latency
Xu <i>et al.</i> [34]	PEKS	Inv-Enc	Multi	Dynamic	None	Heavy PK cost
Li <i>et al.</i> [29]	PEKS	Inv-Enc	Single	Dynamic	Cloud	Complex updates
CryptoCloud	SSE-based	Inv-Enc	Single/Multi	Dynamic	TTP	Trusted TTP

TABLE 2 | Adversarial Behavior of Each Entity

Entity	Assumption	Behavior Description
TTP	Trusted	Manages secret keys and authorizations. If compromised, it may leak keys and identity mappings, resulting in a complete confidentiality breach.
CSP	Untrusted	Executes protocols correctly but may attempt to infer private information from encrypted data, indexes, or queries.
DU	Semi-trusted	Follows the protocol but may misuse authorized access, replay authorized queries, or collude with other users.
AD	Distrusted	Attempts to eavesdrop, access, or alter data stored in the CSP or accessed by the DU. May observe stored data or data in transit but is limited by TLS protections.

to plaintext data. The AD tries to gain access to the documents stored in the CSP, which are not shared with the AD.

Assumptions: Secure by Default

We adopt the following baseline assumptions to reflect practical cloud deployments and bound the adversary’s capabilities.

- **Secure Communication:** TLS is used for all communication between entities to avoid tampering or eavesdropping.
- **Authentication:** The DU’s confirmed email address, verified by the TTP, is linked to a folder that the DO shares data access with. The DO and DU authenticate to the TTP and CSP.
- **Cryptography:** Data and indexes are encrypted using AES, and the distribution of decryption keys is secured using RSA.
- **Key Management:** Strongly random keys are generated, never sent in plaintext, and securely stored.
- **Trusted TTP:** By default, the TTP is considered trustworthy, except for the specific compromise scenario described below.

Adversarial Behavior Summary

In order to make duties and risks more straightforward, Table 2 lists the presumed level of trust for each entity along with the types of misbehaviour they might display. This summary adds

value to the threat model by clearly illustrating how those pre-suppositions translate into actual hostile actions.

Under these assumptions, the primary threats arise from inference attempts by the CSP, misuse by authorized users, or compromise of the TTP.

4 | Security Requirements

Under the specified threat model, the proposed solution aims to ensure that outsourced data remains safe. The following are the main security requirements:

- **Data Confidentiality:** All outsourced documents must remain secret from unauthorized parties. This includes the CSP, which is presumed to be unreliable, as well as external adversaries. The CSP should not be able to retrieve or deduce plaintext material, as it only stores encrypted files and indexes. The TTP verifies that only the DO and authorised DUs can access plaintext data and acquire the required decryption keys.
- **Index Privacy:** Keyword-based enquiries over encrypted documents are made possible by the searchable index, but sensitive metadata must not be revealed. Specifically, the CSP should not be able to infer the index’s internal structure, the number of documents associated with each keyword, or the actual keywords. By protecting index privacy, correlations between keywords and documents cannot be inferred by statistical or frequency analysis.
- **Keyword Privacy:** The CSP must not be able to see the search queries issued by the DO or authorised DUs. The

CSP should not be able to determine which keyword is being searched for, even when it handles search requests and provides the relevant encrypted results. Patterns that could jeopardise privacy should not be shown by repeatedly searching for the exact phrase. By protecting keyword privacy, the CSP is prevented from creating user-interest or search-habit profiles.

- **Authentication:** Strong authentication is required to ensure that only legitimate entities participate in the system. The TTP is responsible for authenticating both DOs and DUs before issuing search tokens or decryption keys. Similarly, communication among the DO, DU, TTP, and CSP must take place over mutually authenticated secure channels (e.g., TLS) to prevent impersonation or man-in-the-middle attacks. This guarantees that both entities and communication links are trustworthy.
- **Authorization:** The TTP is responsible for ensuring that DUs explicitly approved by the DO receive access privileges. Unauthorized users must be denied the ability to perform searches or obtain decryption keys. Revoked users lose authorization immediately.
- **Access Control:** Access to encrypted documents and search functionalities must remain under the full control of the DO. Fine-grained policies determine which DUs are permitted to search or retrieve specific data. Even though files are stored on an untrusted CSP, access control ensures that only authorized entities can interact with the data.
- **Revoked Share:** A DU's search and decryption capabilities must be instantly removed from the user if their authorization is revoked. Revoked users must not be able to access future data by using previously issued keys, tokens, or query results. If required by policy, forward and backward-secure techniques should be used to guarantee that revocation applies to both recently acquired data and previously outsourced information. This ensures that access control and data confidentiality are upheld even in the presence of revoked users.

5 | Proposed Framework

CryptoCloud, the proposed framework, creates a secure system for storing, searching, and sharing encrypted data in the cloud. The design facilitates effective retrieval and user-controlled sharing while guaranteeing data confidentiality and query privacy. The basic idea behind CryptoCloud is to encrypt files and their indexes before sending them to the cloud. Through the TTP, search queries are converted into encrypted search terms, allowing the cloud service provider to deliver only ciphertexts that match the query without learning the keywords.

The TTP is not merely a key escrow that duplicates the DO; it is required by the multi-user and revocation goals. If the DO shared the keyword secret sk directly with a DU, that DU could generate valid search tokens offline indefinitely, and revocation would be impossible without re-encrypting the entire index. By mediating every token request, the TTP ensures that (i) a DU never learns sk and therefore cannot search once revoked, (ii) search and key-release remain available even when the DO is offline, and (iii)

new users are enrolled non-interactively without involving existing users. The DO retains policy control, while the TTP is the always-available enforcement point that makes non-interactive authorization and instant revocation feasible.

An outline of the proposed CryptoCloud system architecture is shown in Fig. 1, which also shows how the key entities interact and how their data flows securely. We then outline the scenario-based workflows for the DO and DU, providing a summary of the notations used to depict the system clearly.

5.1 | Notation

Before presenting the details of the proposed model, it is useful to introduce the notations used throughout this section. Table 3 summarizes these notations, including files and identifiers, cryptographic keys and key management, encrypted keywords, indexing structures, search and retrieval functions, and sharing operations. These definitions provide a clear reference for understanding the workflows described in the following subsections.

5.2 | Scenario 1: Data Owner Searchable Encryption

This scenario, which focuses on the DO, demonstrates how encrypted data can be outsourced and then securely searched. Scenarios 1(A) and 1(B) for data upload and data retrieval each include two sections. When taken as a whole, these scenarios show how the DO interacts with the CSP and the TTP, ensuring that the CSP uses only ciphertext.

5.2.1 | Scenario 1(A): Data Upload

Prior to outsourcing, DO produces files and searchable keywords during the data upload phase. Keywords are encrypted with a secret key, and each file is encrypted with a different symmetric key. These encrypted files are then stored at the CSP, which stores only ciphertext and cannot read plaintext data, along with the encrypted index. The files are given distinct identities by CSP, which also stores them and the encrypted index for future searches. DO sends the file key mapping table and the secret key to the TTP to facilitate controlled access and secure search. Only authorised DOs can query and retrieve documents, as the TTP encrypts search terms. Once authorized, DO can issue secure search requests to TTP, while CSP continues to operate only on encrypted data. The overall workflow of this process is illustrated in Fig. 2.

5.2.2 | Scenario 1(B): Data Retrieval

In the data retrieval phase, DO first sends a plaintext keyword and the target folder ID to TTP. TTP encrypts the keyword using the secret key and generates an encrypted search keyword, which it returns to DO. Now, DO searches with the encrypted search keyword in the CSP, which searches the encrypted index for matching entries and returns the identifiers of the relevant encrypted files. CSP only works with encrypted data and never learns the actual keyword or file contents.

Next, DO requests the encrypted files from CSP and sends the corresponding file identifiers to TTP. Using its key mapping table, TTP returns the appropriate decryption keys. Finally, DO decrypts the ciphertexts and recovers the original files. Throughout

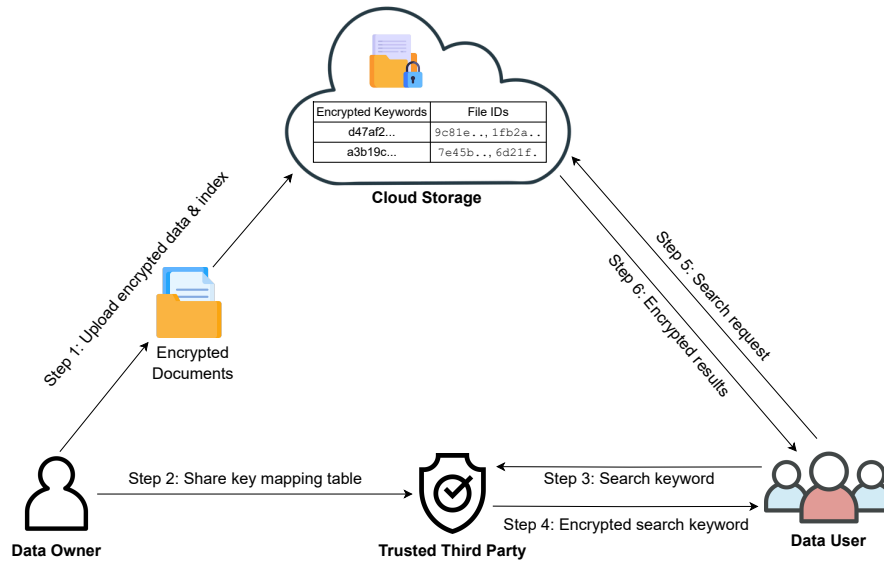


FIGURE 1 | System overview of the proposed CryptoCloud searchable encryption framework.

TABLE 3 | Notations for Searchable Encryption Scenarios

Symbol / Function	Description	Scenario
Files & Identifiers		
f_i	Plaintext file i , $i \in \{1, \dots, n\}$	All
$Enc_{rk_i}(f_i)$	Encrypted file ef_i using symmetric key rk_i	All
FID_i	File ID assigned by CSP for encrypted file	All
$FoldID$	Folder ID where encrypted files are stored	2A, 2B
Keys & Key Management		
rk_i	Random symmetric encryption key for file i , generated by DO	All
sk	Secret key for keyword encryption, generated by DO	All
$\mathcal{M}_{rk}$	Key mapping table: $FID_i \leftrightarrow rk_i$, shared with TTP	1, 2A
PK_{DU}	RSA public key generated by DU	2A, 2B
PR_{DU}	RSA private key generated by DU	2A, 2B
$Enc_{PK_{DU}}(rk_i)$	File key rk_i encrypted under DU's public key	2A, 2B
$Dec_{PR_{DU}}(\cdot)$	Decrypt data using DU's private key	2A, 2B
Keywords & Encryption		
kw_{ij}	Keyword j for file i (plaintext)	All
$Enc_{sk}(kw_{ij})$	Encrypted keyword ekw_{ij} for file i	All
kw_s	Search keyword provided by user (plaintext)	All
$Enc_{sk}(kw_s)$	Encrypted search keyword ekw_s	All
Indexing		
I	Inverted index: keyword $\mapsto \{FID\}$	All
$Enc_{sk}(I)$	Encrypted inverted index I'	All
Search & Retrieval Functions		
$T(kw_s, sk)$	Encrypted search keyword function: encrypts kw_s to get ekw_s	All
$S(ekw_s, I')$	Search in encrypted index to get F_w	All
F_w	Set of matching File IDs	All
Sharing & Registration		
Share(DU, FoldID)	Share folder with DU (via their registered email)	2A, 2B
RegLink(DU)	Registration link sent to DU for account creation	2B only
Verify(DU)	Verify DU account existence	2A, 2B

this process, CSP only handles encrypted data, while TTP ensures that only authorized search and retrieval requests are carried out. The workflow of this phase is depicted in the Fig. 3.

5.3 | Scenario 2: Data User Searchable Encryption

This scenario focuses on the DU and demonstrates how encrypted data shared by the DO can be securely accessed. Scenario

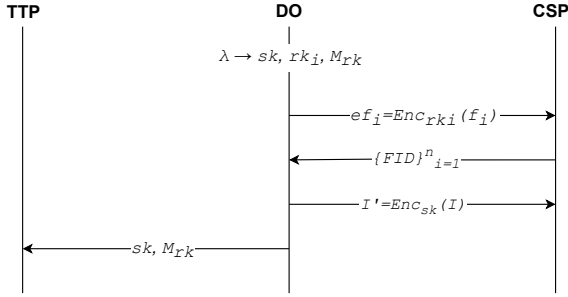


FIGURE 2 | Flow diagram of DO data upload.

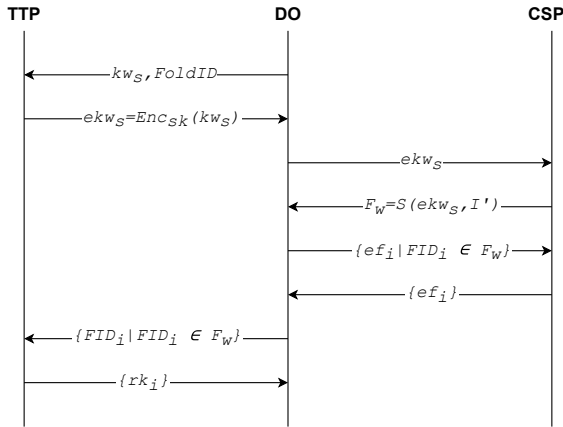


FIGURE 3 | Flow diagram of DO data retrieval.

2(A) for registered DU retrieval and Scenario 2(B) for unregistered DU retrieval are the two sections. When combined, these scenarios demonstrate how the DU works with the CSP and TTP to retrieve authorised files and conduct secure searches without disclosing unencrypted information. The ensuing subsections provide the specifics.

5.3.1 | Scenario 2(A): Data Retrieval by Registered Data User

Here, DU searches over the data that DO has supplied. DU and DO first share a folder with several files, and TTP confirms DU's identity and permissions to view the folder. DU sends a folder identifier and a keyword to TTP to start a search, and TTP uses the secret key to create an encrypted search query, then sends it back to DU. After that, CSP receives this encrypted search keyword from DU, verifies the encrypted index, and provides the identifiers of the corresponding encrypted files.

After that, DU obtains the encrypted data from the CSP and requests the matching decryption keys from the TTP. TTP encrypts the file keys using DU's RSA public key before returning them, ensuring secure transmission. DU uses its private key to recover the file keys and then decrypts the files locally. In this way, CSP only sees encrypted data, while TTP ensures authentication and secure key distribution. The flow diagram illustrating this process is shown in Fig. 4.

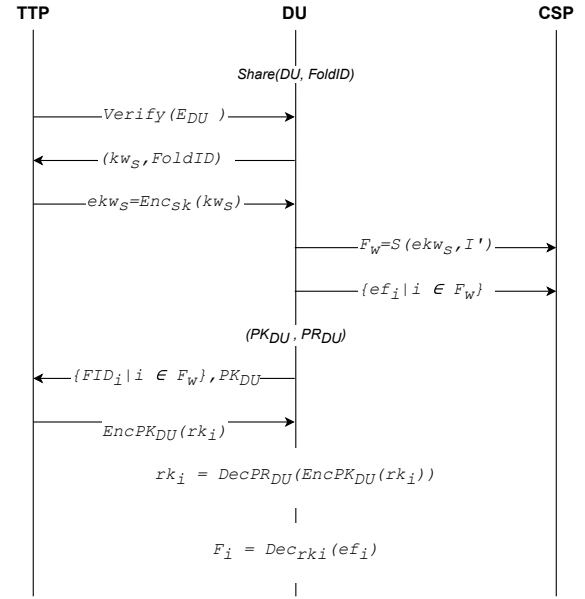


FIGURE 4 | Flow diagram of Data Retrieval by Registered Data User.

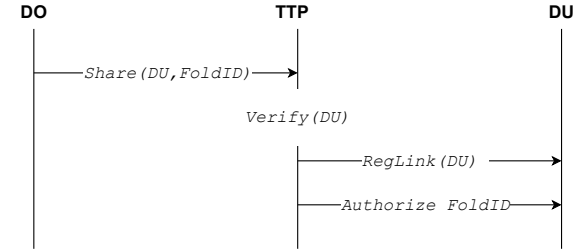


FIGURE 5 | Flow diagram of Data Retrieval by Unregistered Data User

5.3.2 | Scenario 2(B): Data Retrieval by Unregistered Data User

In this scenario, DU does not register in the system beforehand. When DO shares a folder with DU using an email address, TTP checks whether the DU is already registered. If no account exists, TTP sends a registration link to the DU's email. The DU completes the registration process, after which TTP verifies the DU's identity and authorizes access to the shared folder at CSP as shown in Fig. 5.

Once registration is complete, the retrieval steps are the same as in Scenario 2(A). DU requests an encrypted search keyword from TTP, submits it to CSP for a secure search, retrieves the encrypted files, and then obtains the required decryption keys from TTP using an RSA key pair. This guarantees that TTP manages key management and authentication, while CSP only works with encrypted data.

6 | Implementation

This section describes how the proposed framework was realized as CryptoCloud, a full-stack web application that demonstrates end-to-end, privacy-preserving keyword search over encrypted documents stored in a public cloud environment. We summarize the architecture, the evaluation dataset, and the design principles; an extended walkthrough of the user interface is omitted here for conciseness.

The two primary elements of the CryptoCloud system are a stateless backend service and a browser-based frontend application, communicating through JSON-based RESTful APIs. To ensure that passphrases, keys, and plaintext data are never revealed to the cloud service provider (CSP) or the backend, all cryptographic operations are performed on the client side. The CSP is Google Drive, and key wrapping, authorisation, and the issuance of encrypted search keywords are managed by a lightweight trusted third-party (TTP) backend.

6.1 | Data Selection

We chose data from electronic health records (EHRs) to assess our deployment. Beyond standard charts, EHRs include diagnoses, prescriptions, treatment plans, imaging, test results, and allergies. They are patient-centered and real-time, allowing authorised users to safely access various healthcare organisations, including pharmacies, emergency rooms, and labs. Although cloud-based EHR systems increase efficiency and interoperability, they also pose security risks, particularly through unauthorized access and data breaches; encryption and role-based access control are therefore essential both in transit and at rest. The EMRBots dataset [41], comprising 100,000 virtual patients with 361,760 admissions and 107,535,387 laboratory observations, was used for testing. The data is appropriate for assessing secure searchable encryption because, despite being synthetic, it faithfully captures the volume and complexity of real-world records.

6.2 | Frontend Application

The *CryptoCloud Frontend* is a cross-platform web application that runs entirely on the user's device in a standard browser, on both desktop and mobile. Unlike traditional web applications that could inadvertently disclose sensitive information, such as passphrases or cryptographic keys, to remote servers, the proposed approach enforces client-side cryptographic operations throughout the workflow: unencrypted data and raw keys never leave the user's context. The application's modular architecture improves maintainability, asynchronous data processing keeps the interface responsive during cryptographic or I/O operations, and the interface exposes secure encryption, search, and sharing functions without requiring technical expertise from the user.

6.2.1 | User Workflows

We summarize the principal user-facing workflows; each maps directly onto the protocol scenarios of Section 5.

- **Account management and storage consent:** Users register with a verified email address and a strong passphrase

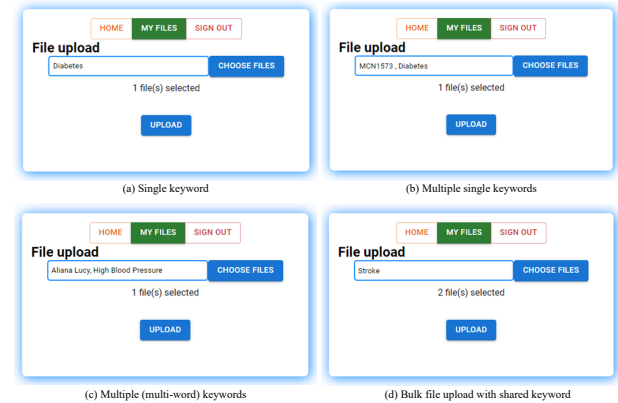


FIGURE 6 | Upload modes supported in *CryptoCloud*: (a) single keyword; (b) multiple single keywords; (c) multiple keywords (phrases); and (d) bulk upload under a shared keyword set.

(minimum eight characters mixing cases, digits, and special characters); weak passphrases and duplicate registrations are rejected, and only salted password hashes are stored for authentication. The workflow is identical for DO and DU. After login, the application requests access to the user's Google Drive through Google's OAuth 2.0 authorization flow, so CryptoCloud never sees the user's Google credentials. This consent is granted once; access tokens are subsequently managed by Google's API, and storage access is confined to the client-side application—the backend never handles authentication secrets or unencrypted data.

- **Folder management:** After login, the DO is presented with the folders retrieved from Google Drive, each offering three core operations—Upload, Share, and Search. A notification panel provides real-time status feedback (e.g., key generation, upload progress, index updates), and a refresh option synchronizes the view with the cloud, including newly added or deleted files.
- **Upload:** The upload workflow adds files to a selected folder while preserving end-to-end confidentiality and future searchability. The DO selects one or more files and assigns keywords; all cryptographic operations are performed locally in the browser. The workflow proceeds in four steps: (i) a fresh AES symmetric key is generated for each file; (ii) the encrypted files are uploaded to Google Drive over TLS, and a *FID* is returned for each; (iii) encrypted keywords and the corresponding *FIDs* are inserted into the encrypted inverted index; and (iv) the folder view is refreshed. Only encrypted files and encrypted index entries ever reach the CSP. Keywords can be assigned in four modes, illustrated in Fig. 6: (a) a single keyword (e.g., a patient ID or a disease name); (b) multiple comma-separated single keywords, enabling discovery of the same file through several exact matches; (c) multi-word phrases (e.g., *High Blood Pressure*) treated as single keyword units; and (d) bulk upload, associating several files with a common keyword set in a single action. Newly uploaded documents become searchable immediately, and failures (e.g., network errors) are reported without affecting previously uploaded files.

TABLE 4 | Keywords associated with sample EHR documents used to demonstrate search functionality.

Keywords	FID(s)
PID202295894, MCN1573, Diabetes	Patient 1
Aliana Lucy, High Blood Pressure	Patient 2
Diabetes	Patient 3
Stroke	Patient 4, Patient 5

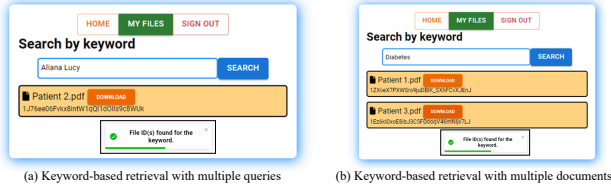


FIGURE 7 | Keyword-based retrieval supporting multiple query terms and multiple matching documents.

- **Share:** To grant a DU access to a folder, the DO enters the DU's email address and confirms the share action. Folder permissions are created internally through Google Drive's permission mechanism for the DU's email address, exposed in CryptoCloud as a single streamlined action, and the DO retains centralized access control without any plaintext disclosure. The designated recipient receives an email invitation; if they do not yet have a CryptoCloud account, the invitation contains a registration link (Scenario 2(B)), and access to encrypted search keywords and wrapped decryption keys is granted only after a verified account is established.
- **Search:** Both the DO and the DU locate encrypted files through keyword queries while the plaintext keywords remain hidden from the CSP. When a query is submitted, an encrypted search keyword obtained from the TTP is evaluated by the CSP against the encrypted inverted index; only the identifiers of matching encrypted files (*FIDs*) are returned, with options to download and decrypt. Table 4 lists the keyword assignments of five sample EHR documents used to demonstrate the supported query types. Single-keyword retrieval covers both direct queries and alternative identifiers: Patient 1 is reachable via its primary identifier (PID202295894), its Medicare number (MCN1573), or its associated condition (*Diabetes*). Multi-keyword retrieval allows a document to be found through any of several query terms (Patient 2 via *Aliana Lucy* or *High Blood Pressure*), and a keyword shared by several documents returns all matches (*Diabetes* returns Patients 1 and 3). Representative results are shown in Fig. 7. If a query matches no document, the interface reports this clearly with a "No file found" notification.

6.3 | Backend Application

The proposed system's server-side component, called *CryptoCloud Backend*, functions as a stateless, lightweight API that

safely handles metadata and enables controlled access to encrypted documents. The backend handles no passphrases or plaintext data, as all cryptographic operations on documents are performed entirely on the client side. Instead, it primarily coordinates metadata, manages authorization records, and communicates with the cloud service provider, ensuring that sensitive information remains isolated from the server.

The backend architecture emphasizes modularity, maintainability, and scalability. It employs a hybrid design pattern that improves code organization and facilitates future growth by separating request management from business logic. Because the backend and frontend frameworks function independently, there are no dependencies between them, and they can grow and change independently.

6.3.1 | Design Principles

The source code structure and database design were deemed essential for creating the CryptoCloud backend. In the source code, we separated request handling from business logic using a hybrid design pattern. This facilitates future additions, enhances organization, and facilitates developer participation. To maintain the separation of concerns, database communication was modularised. To optimise manageability and efficiency, we separated the database into three sections:

- **File Info database** holds properties such as *FoldID*, *FID*, the wrapped symmetric encryption key, and the file name. Additionally, it keeps track of which DO has permission to access each file.
- **User ID database** stores information about the DO and DU, including email addresses, hashed passwords, and Google Drive access tokens.
- **Identifiers database** links user credentials and file metadata, supporting secure retrieval workflows.

This modular design ensures that database operations remain efficient while avoiding performance bottlenecks that could arise from a poorly mixed schema. It also provides a strong foundation for scalability and integration with future system extensions.

7 | Performance Analysis

We evaluated CryptoCloud framework under controlled load on a Windows Server 2025 machine with an AMD 64-core CPU and 64 GB RAM. All interactions among DO, DU, CSP and TTP used HTTPS. We evaluate four metrics such as (a) encryption time for generating Encrypted Search Keywords, (b) search time on cloud storage, (c) search time on local storage, and (d) inverted-index generation time each discussed below. The workload is identical across experiments: concurrent users $U \in \{100, 200, 300, 500, 700, 1000\}$ and keywords per user $K \in \{5, 10, 15, 20\}$. Users start simultaneously to emulate a worst-case burst and expose saturation effects.

Encryption time We isolate the TTP's web service endpoint that transforms a DU's plaintext keyword into an Encrypted Search Keyword and measure the server-side encryption generation time

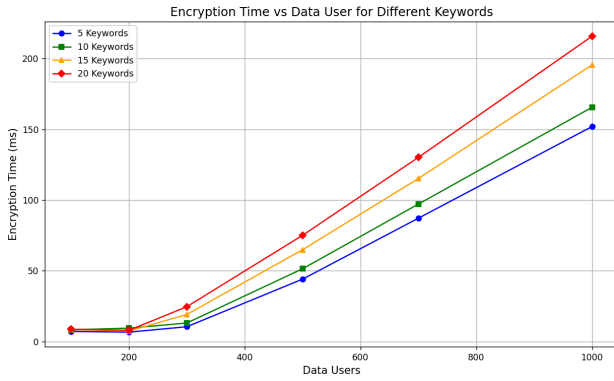


FIGURE 8 | Encryption time of Encrypted Search Keywords

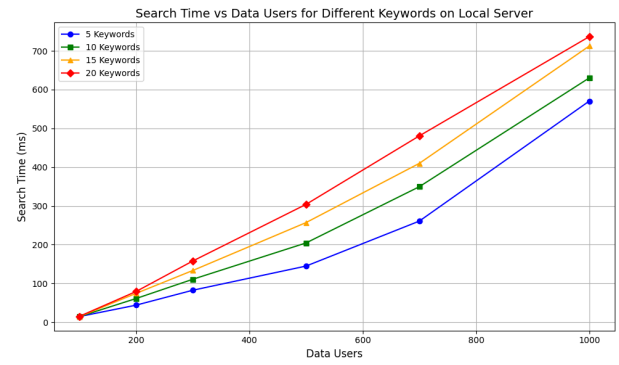


FIGURE 10 | Local search time

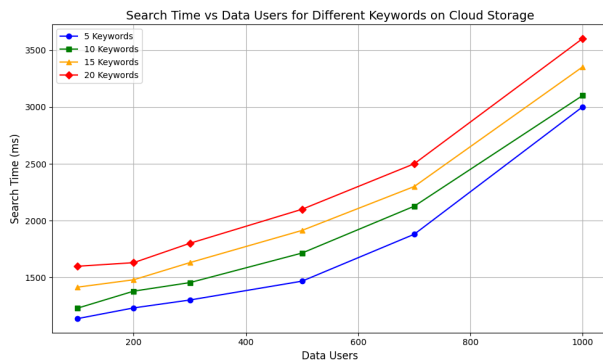


FIGURE 9 | Cloud search time

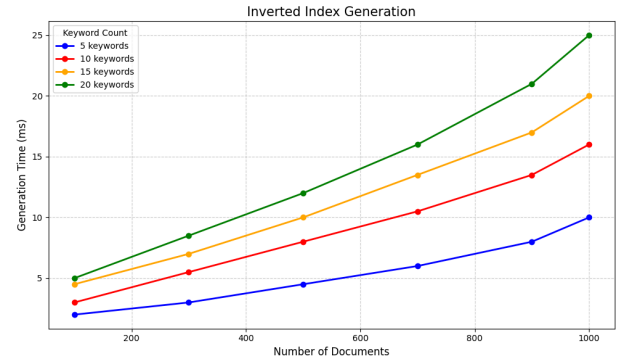


FIGURE 11 | Inverted-index generation time

for this operation. In the protocol framework, for each (U, K) pair, a DU submits plaintext keywords and the TTP returns Encrypted Search Keywords. We measure only server-side generation time (no client rendering or CSP I/O). Encryption time grows roughly with K , remains stable at low to moderate U , and shows queuing-driven increases only at higher concurrency. The encryption time is shown in Fig. 8.

Search time on cloud storageNext, given an Encrypted Search Keyword, the CSP evaluates it over the encrypted inverted index and returns matching ciphertext identifiers. Latency increases with both U and K , with sensitivity to U more pronounced as K grows. The cloud search time is shown in Fig. 9.

Search time on local storageTo factor out CSP and network effects, we repeat the experiments on local storage. Trends with respect to U and K are similar, but absolute latencies are consistently lower than on the cloud. The local search time is shown in Fig. 10.

Comparing the search time of cloud and local storage, the gap quantifies CSP and network overheads rather than cryptographic or indexing costs. The gap is modest at low to moderate concurrency and widens at higher U and K . Nearly parallel slopes indicate predictable scaling of the encrypted lookup.

Inverted-index generation timeWe measure inverted-index generation as the number of documents grows for different keyword counts. Build time increases with the total number of keyword-document pairs produced by (U, K) . Because index creation is performed at data-preparation time and supports incremental updates, this cost is amortized across subsequent searches. The inverted-index generation time is shown in Fig. 11.

Across all experiments, encrypted search keyword generation is lightweight and stable under moderate concurrency; end-to-end search latency on cloud storage is primarily influenced by CSP/network overhead relative to local storage; and inverted-index generation scales with documents and keywords. Overall, CryptoCloud maintains practical responsiveness under realistic multi-user loads while preserving end-to-end confidentiality.

8 | Security Analysis

This section presents a security analysis of the proposed framework and demonstrates that it satisfies the security requirements defined earlier. We assume an untrusted CSP; a minimally trusted TTP that authenticates DO and DU, issues deterministic encrypted search keywords, and holds the encryption keys; and a backend database server that stores user passphrases for authentication only. Documents and per-file indexes are encrypted prior

to outsourcing to the CSP, and all inter-component communications occur over secure channels. Below, we show how the design meets each stated requirement.

- **Data confidentiality:** All documents are encrypted on the client side before upload, and the index is stored only in encrypted form. Encryption keys are maintained by the TTP; passphrases are kept at the web application database server for authentication. The CSP therefore observes only ciphertexts and cannot infer document content. Only the DO and authorized DU after authorization receive the wrapped keys required to decrypt locally.
- **Index privacy:** The DO selects plaintext keywords for each file, encrypts those keywords, and uploads the resulting encrypted keywords to the CSP as part of the inverted index that maps encrypted keywords to encrypted file identifiers. Because the CSP sees only encrypted keywords and encrypted identifiers, it cannot read or infer the underlying keywords or document contents from the index.
- **Keyword privacy:** Queries are issued using deterministic encrypted search keywords generated by the TTP. For a given encrypted keyword, the encrypted search keyword is stable across sessions; repeated searches therefore produce the same token. While this allows the CSP to test token equality, it still cannot recover the underlying keyword or any document content and learns only whether the token matches entries in the encrypted index. We are explicit about the residual leakage: because search tokens are deterministic, the CSP observes the search pattern (repeated queries produce identical tokens) and the access pattern (which encrypted identifiers match). Hiding these patterns is possible—e.g., via ORAM or the dual-server and pattern-hiding techniques of Wang *et al.* [42]—but at a cost that conflicts with the lightweight, single-provider overlay model targeted here. We therefore treat access- and search-pattern hiding as an explicit design trade-off rather than an oversight.
- **Authentication:** The framework enforces DO-defined access control at the TTP. The DO specifies which DUs may search or decrypt specific folders/documents; the TTP checks this policy before servicing any request. Only DUs explicitly approved by the DO receive authorization artifacts: the TTP issues deterministic encrypted search keywords and releases wrapped decryption keys only when the DO's policy permits it.
- **Access control:** The DO retains full control over who may search and decrypt. The CSP returns only encrypted documents. Performing a search requires a valid encrypted search keyword, and decryption requires the corresponding wrapped file key—i.e., the per-document symmetric key encrypted under the DU's public key and issued by the TTP. Consequently, even if a DU downloads an encrypted file, they cannot recover plaintext without unwrapping this file key with their private key. Fine-grained, DO-defined policies at the TTP govern both encrypted search keyword issuance and wrapped-key release. Note that wrapping under the DU's public key is not redundant with TLS: TLS protects data only in transit and terminates at the backend, whereas the wrapped key $\text{Enc}_{c_{PK_{DU}}}(rk_i)$ remains encrypted at rest in

the metadata store and end-to-end from the TTP to the intended DU, so that no backend process, TLS-terminating proxy, or database compromise can recover the file key.

- **Revocation:** When a DU is revoked, the TTP immediately stops issuing encrypted search keywords and wrapped file keys to that user. Because encrypted search keywords are obtained from the TTP at query time, a revoked DU cannot perform new searches; and without new wrapped file keys, the user cannot decrypt subsequently retrieved documents.

We note a deliberate scope boundary consistent with our threat model: an authorized DU receives the per-file symmetric key and can therefore decrypt locally. Preventing a semi-trusted DU from redistributing recovered keys or plaintext is outside the guarantees of the present design. Proxy re-encryption is a natural extension that would let the TTP re-encrypt file keys toward each DU without ever exposing a reusable raw key, which we identify as a direction for future work (Section 9).

Taken together, encrypted-at-rest storage on the CSP, encrypted keyword indexing, DU-initiated encrypted search keyword-based search using TTP-issued tokens, authenticated and authorization-scoped release of wrapped file keys, and immediate revocation provide end-to-end protection under the stated threat model and demonstrate that the framework achieves its intended security guarantees.

8.1 | Formal Verification using Scyther

To formally validate the correctness and confidentiality of the message exchanges in the proposed CryptoCloud framework, we performed protocol verification using Scyther [43]. Scyther is an automated security protocol analysis tool widely used in applied cryptography and protocol verification. It operates under the Dolev-Yao adversarial model, in which the adversary has full control over the communication network and may intercept, replay, or modify messages, but cannot break cryptographic primitives. The modeled protocol captures the end-to-end interaction among DO, TTP, CSP, and DU during encrypted keyword search and controlled file decryption. Two fundamental security attributes that complement our system goals are the focus of the verification:

- **Confidentiality:** Secrecy of plaintext search terms, per-file encryption keys, and keyword encryption keys
- **Authorization:** Guarantee that the decryption keys for retrieved ciphertexts can only be obtained by authorised and authenticated users.

Scyther reported no attacks for all specified secrecy and authorization assertions after analysing the protocol under unbounded session executions. The study verifies explicitly the confidentiality of: (i) the symmetric encryption keys (rk) that are utilised for every file; (ii) the encryption key for keywords (sk); and, lastly, (iii) the plaintext query keywords (kw). Also, it confirms that only authorized DUs release decryption keys, in compliance with the access-control policy imposed by the TTP. These results provide a formal confirmation of the security properties discussed earlier in this section.

Author Contributions

Md Mazharul Islam: conceptualization, methodology, formal analysis, investigation, validation, visualization, writing – original draft **Md. Shahadul Alam Patwary:** software, implementation, validation **Salekul Islam:** supervision, methodology, writing – review and editing. **Rajesh Palit:** supervision, methodology, writing – review and editing.

All authors read and approved the final manuscript.

Conflicts of Interest

The authors declare no conflicts of interest.

References

- Mathur P. Cloud computing infrastructure, platforms, and software for scientific research. *High Performance Computing in Biomimetics: Modeling, Architecture and Applications*. 2024;89–127.
- Mell P, Grance T, others. The NIST definition of cloud computing. 2011.
- Ma Q, Qin Y, Zhu C, Gao L, Chen X. Joint resource trading and task scheduling in edge-cloud computing networks. *IEEE Transactions on Networking*. 2025.
- Chippagiri S. A Study of Cloud Security Frameworks for Safeguarding Multi-Tenant Cloud Architectures. *International Journal of Computer Applications*. 2025;975:8887.
- Arif T, Jo B, Park JH. A Comprehensive Survey of Privacy-Enhancing and Trust-Centric Cloud-Native Security Techniques Against Cyber Threats. *Sensors*. 2025;25(8):2350.
- Jin H, Luo Y, Li P, Mathew J. A review of secure and privacy-preserving medical data sharing. *IEEE access*. 2019;7:61656–61669.
- Andola N, Gahlot R, Yadav VK, Venkatesan S, Verma S. Searchable encryption on the cloud: a survey. *The Journal of Supercomputing*. 2022;78(7):9952–9984.
- Chandwani G, Ahlawat A, Dubey G. An approach for document retrieval using cluster-based inverted indexing. *Journal of Information Science*. 2023;49(3):726–739.
- Sharma D. Searchable encryption: A survey. *Information Security Journal: A Global Perspective*. 2023;32(2):76–119.
- Cai C, Weng J, Yuan X, Wang C. Enabling reliable keyword search in encrypted decentralized storage with fairness. *IEEE Transactions on Dependable and Secure Computing*. 2018;18(1):131–144.
- Yang X, Wang Q, Qi S, Li K, Qi Y. RO (SE) 2: Search-Efficient Robust Searchable Encryption with Forward and Backward Security. *IEEE Transactions on Computers*. 2025.
- Yang Y, Fan H, Zhang J, Li B, Ma H, Gu X. SMSSE: Size-pattern mitigation searchable symmetric encryption. *IEEE Transactions on Information Forensics and Security*. 2025.
- Patwary MSA, Palit R. A Prototype of a Secured File Storing and Sharing System for Cloud Storage Infrastructure. In: IEEE. 2019:1–8.
- Song DX, Wagner D, Perrig A. Practical techniques for searches on encrypted data. In: IEEE. 2000:44–55.
- Curtmola R, Garay J, Kamara S, Ostrovsky R. Searchable symmetric encryption: improved definitions and efficient constructions. In: ACM. 2006:79–88.
- Boneh D, Di Crescenzo G, Ostrovsky R, Persiano G. Public key encryption with keyword search. In: Springer. 2004:506–522.
- Chaudhari P, Das ML. Privacy preserving searchable encryption with fine-grained access control. *IEEE Transactions on Cloud Computing*. 2019;9(2):753–762.
- Islam M, Palit R. A Keyword Based Searching and Sharing Scheme on the Encrypted Cloud Data. In: IEEE. 2023:1–6.
- Li S, Jing X, Wang Y, Xu X, Zhang Z, Wang J. Practical searchable encryption scheme against response identity attacks. *Information Sciences*. 2025;706:121975.
- Liu H, Xu L, Liu X, Mei L, Xu C. Query Correlation Attack Against Searchable Symmetric Encryption With Supporting for Conjunctive Queries. *IEEE Transactions on Information Forensics and Security*. 2025.
- Bian B, Xu Q, Liu J, Wang J. Practical Robust Dynamic Searchable Symmetric Encryption Supporting Conjunctive Queries. In: Springer. 2025:63–83.
- Hu Z, Wang J, Chen Z, et al. SEAC: dynamic searchable symmetric encryption with lightweight update-search permission control. *Cybersecurity*. 2025;8(1):75.
- Deng Q, Chen L, Zhu Y, Mu Y. Leakage Reduced Searchable Symmetric Encryption for Multi-keyword Queries. *IEEE Transactions on Cloud Computing*. 2025.
- Ma J, Peng T, Bei G, Waqas M, Alasmay H, Chen S. Efficient privacy-preserving conjunctive searchable encryption for Cloud-IoT healthcare systems. *ACM Transactions on Privacy and Security*. 2025;29(1):1–27.
- Chakraborty D, Majumder A, Samajder S. Making searchable symmetric encryption schemes smaller and faster. *International Journal of Information Security*. 2025;24(1):10.
- Khan AN, Naveed A, Mehmood A, Arora D, Khan AUR, Ali J. BloomSec: Scalable and privacy-preserving searchable encryption for cloud environments. *PLoS One*. 2025;20(12):e0336944.
- Ji L, Li J, Zhang Y, Lu Y. Verifiable searchable symmetric encryption over additive homomorphism. *IEEE Transactions on Information Forensics and Security*. 2025.
- Zhao J, Yu J, Yuan X, Liu JK, Zuo C, Cui H. SEARCHAIN: Searchable Encryption As Rewarded-Useful-Work on Blockchain. In: Springer. 2025:368–384.
- Li H, Susilo W, Shen J, Wang C, Cheng L, Huang Q. Proxy-free Public-key Authenticated Updatable and Searchable Encryption for Cloud Storage. *IEEE Transactions on Dependable and Secure Computing*. 2025.
- Xu Y, Cheng H, Li J, Liu X, Zhang X, Wang M. Lightweight Multi-User Public-Key Authenticated Encryption With Keyword Search. *IEEE Transactions on Information Forensics and Security*. 2025.
- Han M, Xu P, Susilo W, Wang W. Dynamic searchable public-key encryption and its application. *Frontiers of Computer Science*. 2026;20(4):2004802.
- Wang P, Wang H, Ouyang F, Yang Y. Secure Fine-Grained Encrypted Keyword Search for Internet of Things. In: ACM. 2025:93–97.
- Hayashi M, Emura K. Security Analysis on a Public-Key Inverted-Index Keyword Search Scheme with Designated Tester. *Cryptology ePrint Archive*. 2025.
- Xu S, Chen X, Guo Y, et al. Lattice-based forward secure multi-user authenticated searchable encryption for cloud storage systems. *IEEE Transactions on Computers*. 2025.
- Goh EJ. Secure indexes. *Cryptology ePrint Archive*. 2003.
- Fugkeaw S, Deevijit J, Ueasathitwong P, Thanyasukpaisal T. EVSEB: Efficient and Verifiable Searchable Encryption with Boolean Search for Encrypted Cloud Logs. *IEEE Access*. 2025.
- Fugkeaw S, Deevijit J. Se-collab: Achieving fine-grained and efficiently verifiable searchable encryption with boolean multi-keyword search for collaborative iiot data sharing. *IEEE Access*. 2025.
- Gao H, Zhang Y, Wang L, Liu Z, Ma Z. A searchable encryption scheme for blockchain-based digital twins. *IEEE Internet of Things Journal*. 2025.
- Ahn K, Kwak C, Koo D, Hahn C, Hur J. Quantifying Security in Volume-Hiding Searchable Symmetric Encryption Schemes With a Novel Scoring Metric. *IEEE Access*. 2025.
- Miao Y, Ma J, Liu X, Zhang J, Liu Z. VKSE-MO: Verifiable keyword search over encrypted data in multi-owner settings. *Science China Information Sciences*. 2017;60(12):122105.

41. Kartoun U. EMRBots: a 100,000-patient database. figshare, Dataset; 2018
42. Wang X, Ma J, Liu X, et al. Search me in the dark: Privacy-preserving boolean range query over encrypted spatial data. In: IEEE. 2020:2253–2262.
43. Cremers CJ. The scyther tool: Verification, falsification, and analysis of security protocols: Tool paper. In: Springer. 2008:414–418.